

```
#include <iostream>
#include <vector>
#include <omp.h>
#include <cstdlib>
#include <climits> // Include for INT_MAX, INT_MIN
#include <chrono>

using namespace std;
using namespace std::chrono;

int main() {
    int n = 1000000; // Size of array
    vector<int> arr(n);

    // Generate random numbers
    #pragma omp parallel for
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 1000; // Random values between 0-999
    }

    int minVal = INT_MAX;
    int maxVal = INT_MIN;
    long long sum = 0; // Use long long for large sums
    double avg = 0.0;

    // **Start Timing**
    auto start = high_resolution_clock::now();

    // **Parallel Reduction**
```

```

#pragma omp parallel for reduction(min:minVal) reduction(max:maxVal) reduction(+:sum)
for (int i = 0; i < n; i++) {
    minVal = min(minVal, arr[i]); // Corrected syntax
    maxVal = max(maxVal, arr[i]); // Corrected syntax
    sum += arr[i];
}

// **Compute Average**
avg = static_cast<double>(sum) / n;

// **Stop Timing**
auto stop = high_resolution_clock::now();
auto duration = duration_cast<milliseconds>(stop - start);

// **Print Results**
cout << "Parallel Reduction Results:\n";
cout << "Min: " << minVal << endl;
cout << "Max: " << maxVal << endl;
cout << "Sum: " << sum << endl;
cout << "Average: " << avg << endl;
cout << "Execution Time: " << duration.count() << " ms\n";
return 0;
}

```

O/P:

Parallel Reduction Results:

Min: 0

Max: 999

Sum: 496875044

Average: 496.875

Execution Time: 38 ms